LAB REPORT 06

For Loops & Nested Loops in C++

Computer Programming · UET Nowshera · Electrical Engineering

STUDENT	M.Waleed Javed	REG NO	25NWELE0749
SECTION	A	IDE USED	DEV-C++

1 Objective

This lab session focused on gaining practical experience with loop constructs in C++. The following objectives were targeted:

- Understanding the concept and syntax of the for loop in C++
- Learning how to implement nested for loops for structured and repeated operations
- Applying loops to solve problems such as counting, pattern printing, and mathematical computations

2 Theory

2.1 The for Loop

A for loop is a control structure that runs a block of code repeatedly as long as a given condition holds true. It is best suited to situations where the number of iterations is known in advance. The loop is made up of three parts written inside the parentheses: an initialization that runs once at the start, a condition that is checked before every iteration, and an update expression that executes after each pass through the body.

Syntax:

```
for (initialization; condition; update) {
    // statements to execute
}
```

Flow of execution: the initialization runs first, then the condition is evaluated. If it is true, the loop body executes, followed by the update expression, and then the condition is checked again. This cycle continues until the condition becomes false, at which point execution moves past the loop.

2.2 Nested for Loops

A nested loop is simply a loop placed inside another loop. For every single iteration of the outer loop, the inner loop runs through its complete cycle. This pattern is particularly useful when working with twodimensional data such as tables, grids, and printed patterns.

3 Programs Implemented

3.1 — Countdown from 5 to 1

A for loop was used to count down from 5 to 1, printing each value on its own line. After the loop finishes, a message is printed to confirm that execution has moved past the loop body.

```
#include <iostream>
using namespace std;

int main() {
    // initialise i = 5; loop while i > 0; decrement each
iteration     for (int i = 5; i > 0; i--) {         cout << i <<
endl;
    }     cout << "outside loop now" <<
endl;     return 0;
}
```

Output: 5 4 3 2 1 outside loop now

3.2 — Prime Number Check

The user enters a number, and a for loop tests every integer from 2 up to one less than that number to see if any of them divide it evenly. A flag variable tracks whether a divisor was found. Once the loop finishes, the flag determines whether the number is reported as prime or not.

```
#include <iostream>
using namespace std;

int main() {     int num;
cout << "Enter a number: ";
cin >> num;

    int flag = 0; // 0 = prime until proven otherwise

    for (int i = 2; i < num; i++) {         if (num %
i == 0) {  // divisible — not prime             flag
= 1;             break;
        }
    }
```

```
    if (flag == 1)         cout << num << " is not a 
prime number" << endl;     else
        cout << num << " is a prime number" << endl;

    return 0;
}
```

3.3 — Nested Loop Pattern Printing

An outer loop counts down from 7 to 1. For each value of the outer counter, an inner loop prints the numbers 1 through that value separated by spaces. A newline after the inner loop moves to the next row, producing a descending triangle pattern.

```
#include <iostream> using
namespace std;

int main() {
    for (int num = 7; num > 0; num--) {  // outer: rows
for (int j = 1; j <= num; j++) {  // inner: values per row
cout << j << " ";
        }
cout << "\n";
    }
    return 0;
}
```

Output: 1 2 3 4 5 6 7 / 1 2 3 4 5 6 / 1 2 3 4 5 / 1 2 3 4 / 1 2 3 / 1 2 / 1

4 Lab Tasks

4.1 — While Loop: Display Numbers from 1 to 10

A while loop was used to print the integers 1 through 10, one per line. A counter variable is initialised before the loop, checked in the condition, and incremented at the end of each iteration.

```
#include <iostream> using
namespace std;

int main() {     int i =
1;     while (i <= 10) {
cout << i << endl;
i++;
    }
return 0;
}
```

4.2 — Multiplication Table Using Nested For Loops

Two nested for loops generate a 10×10 multiplication table. The outer loop steps through row numbers 1 to 10, and the inner loop steps through column numbers 1 to 10. Each cell prints the product of the row and column indices, separated by a tab character.

```
#include <iostream> using
namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {       // rows
for (int j = 1; j <= 10; j++) {   // columns
cout << i * j << "\t";
        }
cout << endl;
    }
    return 0;
}
```

4.3 — Sum of Numbers from 1 to N

The user enters a positive integer N. A for loop accumulates the sum of all integers from 1 up to N by adding the loop counter to a running total on each iteration. The final sum is displayed after the loop.

```
#include <iostream> using
namespace std;

int main() {     int n, sum = 0;     cout
<< "Enter a positive integer: ";     cin
>> n;

    for (int i = 1; i <= n; i++) {
sum += i;
    }

    cout << "Sum = " << sum << endl;
return 0;
}
```

4.4 — Triangle Pattern Printing

Nested for loops are used to print a right-angled triangle made of asterisks. The outer loop controls the number of rows, and the inner loop prints one asterisk per column up to the current row number. Each row ends with a newline.

```
#include <iostream> using
namespace std;

int main() {     int n;     cout <<
"Enter number of rows: ";     cin >>
n;

    for  (int  i  =  1;  i  <=  n;  i++)  {
for  (int  j  =  1;  j  <=  i;  j++)  {
cout << "* ";
        }
cout << endl;
    }
    return 0;
}
```

5 Conclusion

This lab gave hands-on experience with for loops, while loops, and nested loops in C++. Working through the tasks made it clear how loop constructs control program flow by repeating a block of code for a set number of iterations or while a condition holds. The practical exercises covered a range of applications — counting down, checking primality, generating a full multiplication table, accumulating a sum, and producing printed patterns — all of which demonstrate how loops are one of the most versatile and frequently used tools in any C++ program.